

AURORA-Q: Asynchronous Unified Resource Optimizer for Quantum Simulation on HPC System

Changjong Kim*, Alex Sim†, Kesheng Wu†, Houjun Tang†, Yongseok Son‡, Jisung Park§, and Sunggon Kim*

*Seoul National University of Science and Technology, Seoul, Republic of Korea

†Lawrence Berkeley National Laboratory, Berkeley, CA, USA

‡Chung-Ang University, Seoul, Republic of Korea

§Pohang University of Science and Technology, Pohang, Republic of Korea

Abstract—This paper presents **AURORA-Q**, a tiered-memory quantum circuit simulation framework designed to overcome the memory bottleneck and scalability limits of large-scale quantum circuit simulation on Perlmutter. Its primary strategy is to extend beyond GPU-centric designs bounded by VRAM and introduce a unified tiered-memory architecture that integrates GPU VRAM, DRAM, and storage into a simulation framework. With this design, **AURORA-Q** partitions the full state vector into subsets, manages their residency across memory tiers, overlaps computation with data movement through an async execution pipeline, and employs a dynamic two-level cache with adaptive resource control to exploit access locality under constrained resources.

Our evaluation demonstrates that **AURORA-Q** executes up to 43 qubit simulations on 512 GPUs, a scale unattainable by existing state-of-the-art simulators due to limited memory resources. **AURORA-Q** achieves up to $2.91\times$ speedup compared to baselines, while maintaining up to a 79.3% cache hit rate and reducing pipeline stalls by 73% to hide I/O latency.

Index Terms—Quantum Computing, High-Performance Computing, Quantum Circuit Simulation

I. INTRODUCTION

Quantum computing offers a computational model distinct from classical computing by operating on qubits rather than binary bits. Each qubit is represented as a quantum state with probability amplitudes (e.g., $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$), where superposition allows multiple states and entanglement creates correlated interactions between qubits. These properties enable parallelism beyond classical computation [1], [2]. Despite these advantages, current quantum computers remain limited to Noisy Intermediate-Scale Quantum (NISQ) devices with high error rates and short coherence times [3], [4].

To overcome these limitations, High-Performance Computing (HPC) systems have become a practical platform for large-scale quantum simulation, providing sufficient computational resources [5], [6]. Two strategies are commonly used: amplitude sampling and full state vector simulation. Amplitude sampling reduces memory usage by computing only a subset of amplitudes, while full state vector simulation evaluates all amplitudes, providing full accuracy and reproducibility [5], [7]–[9]. Our work focuses on full state vector simulation, as exact results are essential for hardware characterization and algorithm validation.

However, full state vector simulation faces a fundamental scalability challenge. As the number of qubits grows, memory usage increases exponentially, causing severe memory pressure

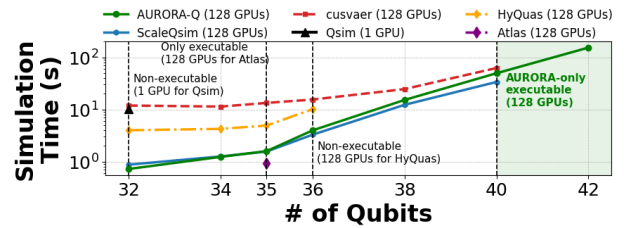


Fig. 1: Scalability of **AURORA-Q** (Proposed), SOTA (*ScaleQsim*, *cusvaer*, *HyQuas*, *Atlas*), and *Qsim* (Existing), executed on 128 GPUs (32 Nodes) using the qft circuits (log-scale).

in large-scale simulations. This is problematic because modern GPUs, despite high computational throughput, provide limited memory (e.g., NVIDIA A100). For example, simulating a 42-qubit state vector (e.g., 2^{42} complex amplitudes, 8 bytes) requires over 32 TB, equivalent to at least 410 GPUs. Beyond storing the state vector, additional overheads further increase memory demand: gate operations, inter- and intra-node communication, and circuit partitioning handling all require extra memory [5], [6], [9]–[11]. Thus, the GPU VRAM footprint exceeds the theoretical minimum, requiring more GPUs and limiting the achievable qubit scale, despite the development of 100+ qubit quantum chips.

Figure 1 shows the scalability comparison of **AURORA-Q** and five SOTA simulators [5], [6], [12]–[14] running qft circuits on 128 GPUs. As depicted, *Qsim* is limited to 32 qubits on a single GPU due to memory constraints, while *cusvaer* and *ScaleQsim* execute up to 40 qubits but fail beyond this point as GPU VRAM becomes insufficient. *HyQuas* shows moderate performance at smaller qubit sizes but fails after 36 qubits due to precompiled kernels. *Atlas* runs only at 35 qubits and fails for other qubit counts because it relies on a fixed qubit–GPU configuration. In contrast, **AURORA-Q** (Our work) executes up to 42 qubits on 128 GPUs, demonstrating higher scalability despite limited GPU resources. These results highlight a key limitation: GPU VRAM alone cannot support large-scale simulation. To overcome this, the system must utilize the entire memory hierarchy, where DRAM serves as a higher-capacity intermediate tier and storage retains the full state vector. This tiered-memory hierarchy extends memory capacity and enables scalable execution.

Many previous studies, as summarized in Table I, have explored works to enhance the scalability of full state vector simulation by addressing memory and computation constraints from architectural and algorithmic perspectives.

TABLE I: Comparison with prior work across three key capabilities: Distributed Parallel Simulation (DPS), Storage Tier Offloading (STO), and Dynamic Cache Management (DCM).

Study	Target Architecture	DPS	STO	DCM
Park <i>et al.</i> [9]	Storage-extended		✓	
Xu <i>et al.</i> [5]	GPU-centric	✓		
Kim <i>et al.</i> [12]	GPU-centric	✓		
Zhang <i>et al.</i> [6]	GPU-centric	✓		
Zhang <i>et al.</i> [10]	Storage-extended		✓	
Guerreschi <i>et al.</i> [15]	CPU-centric	✓		
Wu <i>et al.</i> [16]	CPU-centric	✓		
Li <i>et al.</i> [11]	GPU-centric	✓		
AURORA-Q	Unified tiered memory	✓	✓	✓

Several works [5], [6], [11], [12] adopt distributed parallel simulation, utilizing GPU-optimized kernels and stage-based partitioning to execute large quantum circuits distributed across multiple GPU nodes. Other studies [15], [16] reduce computational costs through CPU-cluster task scheduling or memory compression to manage large-scale simulations within DRAM capacity. In addition, some studies, storage-extended approaches [9], [10] extend memory capacity by offloading the state vector to secondary storage or applying GPU-side compression.

AURORA-Q distinguishes itself from prior works by departing from static resource binding. Most previous studies rely on single-tier GPU VRAM, storage offloading, or GPU-side compression, which constrain simulation to fixed GPU capacity, induce I/O bottlenecks, or consume computational resources for runtime decompression. In contrast, AURORA-Q adopts a decoupled tiered-memory architecture that separates global logical ownership from local physical residency. AURORA-Q uses a global ownership map for static node-level mapping, while each owner node’s local cache controller tracks GPU VRAM, DRAM, and storage residency via cache lists. By exploiting locality, AURORA-Q orchestrates overlapped data movement and computation, ensuring data is resident in the GPU cache on demand and hiding I/O latency.

In this paper, we propose AURORA-Q, a scalable quantum circuit simulation framework designed for leadership-scale HPC systems. Specifically, AURORA-Q (1) partitions the state vector to manage residency across the memory hierarchy beyond GPU VRAM limits, (2) employs an asynchronous execution pipeline that overlaps computation with data movement to hide latency, and (3) integrates a dynamic two-level cache with adaptive resource control to exploit access locality. Our results demonstrate scalable 43-qubit on 512 GPUs, surpassing the memory limits where SOTA simulators fail, and competitive performance with up to $2.91\times$ speedup. AURORA-Q is publicly available at <https://github.com/AURORA-Q-public/AURORAQ.git>.

II. BACKGROUND

A. Full State Vector Simulation

Full state vector simulation is essential for verifying quantum algorithms, as it provides exact results. However, it suffers from performance bottlenecks due to exponential data growth and irregular, non-contiguous memory accesses in gate

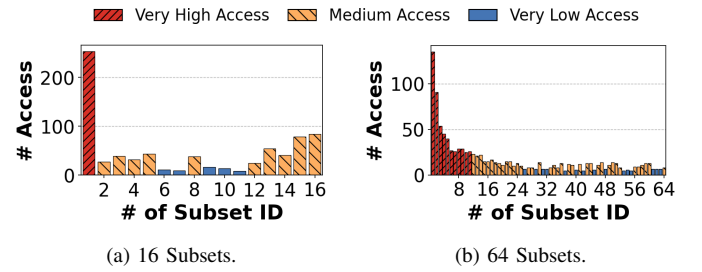


Fig. 2: Skewed subset access pattern in 36-qubit qft circuits.

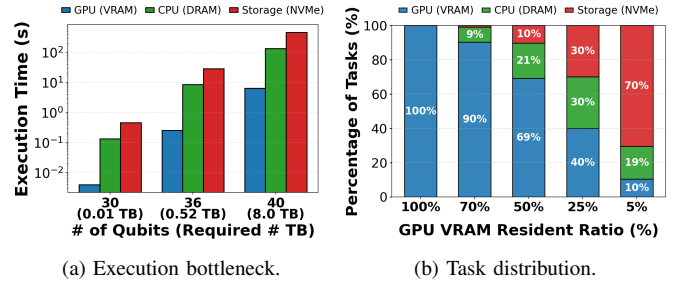


Fig. 3: Performance limits of memory tiers.

operations, making performance highly dependent on memory layout [5], [17], [18]. For example, a single-qubit gate on the k -th qubit accesses amplitude pairs $(\alpha_j, \alpha_{j+2^k})$ separated by a stride of 2^k . Multi-qubit entangling gates generate even more scattered access patterns. These patterns are also non-uniform, as many widely used NISQ-era algorithms (e.g., qft, vqc, vqe) concentrate operations on specific qubits (e.g., control qubits or frequent 1-qubit gates), causing intensive updates to certain memory regions while leaving others idle.

Figure 2 shows the skewed subset access pattern observed in simulations. It profiles the access pattern of a 36-qubit qft circuit partitioned into 16 (e.g., 32 GB) and 64 (e.g., 8 GB) subsets. As depicted, the skew appears in both cases. For the 16 subsets, subset ID 1 accounts for about one-third of all accesses, and the top four subsets cover nearly half, while many subsets contribute only a small fraction. In the 64 subsets, the same hot region is split across finer-grained subsets (e.g., subset 1-12). Each subsets receive fewer accesses, but the aggregate hotspot remains dominant while most subsets remain cold.

While this skewed access pattern is inherent to quantum algorithms, it resembles locality challenges in data-intensive workloads (e.g., graph processing or sparse matrix computations). Some GPU-centric simulators [5], [6] mitigate this effect using pre-compiled kernels and static execution plans to reduce communication overhead, but these strategies are constrained by GPU VRAM capacity. When offloading is required, access skew causes I/O bottlenecks due to repeated high-latency fetches and pipeline stalls. Thus, dynamic data management, latency-hiding pipelines, and locality-aware caching are required to exploit inherent access patterns.

B. Hardware Constraints and Performance Limits

The scalability of full state vector simulation is bounded by memory capacity. Since memory usage grows exponentially, high-performance memory tiers are quickly exhausted. A typical HPC GPU node with four NVIDIA A100 (80 GB)

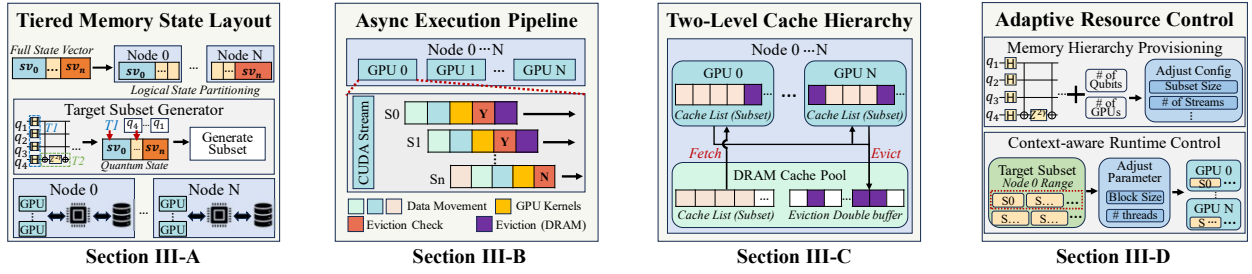


Fig. 4: An overview of our proposed AURORA-Q.

GPUs provides 320 GB, which supports a 35-qubit simulation (256 GB) but is insufficient for 36 qubits (512 GB). Scaling to 40 qubits (8.0 TB) requires offloading state vectors to DRAM and NVMe SSDs, causing notable performance degradation.

Bandwidth Gap and Execution Time. Hierarchical offloading exposes the bandwidth differences across memory tiers. Figure 3a shows the measured throughput of each memory tier using a 1GB micro-benchmark and the measured execution time for various qubit scales. GPU VRAM provides more than 2TB/s of bandwidth (A100 80GB), while PCIe Gen4 and the Lustre parallel file system offer approximately 64GB/s and 25GB/s, respectively. When the state vector exceeds GPU memory capacity (e.g., 40 qubits), execution becomes I/O-bound. This bandwidth disparity causes the GPU to spend most of its cycles waiting for required data rather than computation, increasing total runtime by more than $73.3\times$ compared to an in-memory configuration.

Data Locality and Task Distribution. Performance degradation is driven by data locality and memory transfer cost. Figure 3b shows task distribution as a function of VRAM residency ratio for a 320 GB micro-benchmark on a single node with 4 GPUs. With full VRAM (100%), all tasks run on GPUs, while at 50% the system relies on lower tiers. At a 5% residency ratio, only 10% of tasks use VRAM-resident data, and 70% fetch data from storage, becoming I/O-bound. This shifts the bottleneck from computation to data movement and leaves the GPU idle. These results show that memory capacity alone does not ensure performance. Our focus is to mitigate transfer latency that causes pipeline stalls. This motivates the design of AURORA-Q, which integrates asynchronous execution with dynamic cache management to maximize data locality and compute-I/O overlap.

III. DESIGN OF AURORA-Q

Overview of AURORA-Q's Design. Figure 4 summarizes the key techniques. First, we introduce *Tiered Memory State Layout* (III-A) to partition the full state vector across the memory hierarchy, overcoming GPU VRAM limits. Second, *Async Execution Pipeline* (III-B) overlaps computation with data movement to hide I/O latency. Third, *Two-level Cache Hierarchy* (III-C) optimizes data residency to minimize storage access. Finally, *Adaptive Resource Control* (III-D) dynamically adjusts tasks for optimal resource utilization.

A. Tiered Memory State Layout

Logical State Partitioning. AURORA-Q initiates the layout configuration by calculating the total size of the full state

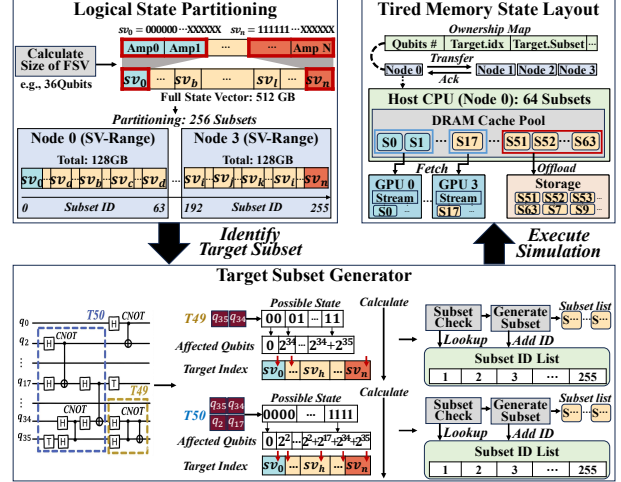


Fig. 5: Architecture and procedure of AURORA-Q.

vector. As shown in *Logical State Partitioning* of Figure 5, a 36-qubit state vector requires a 512 GB logical footprint. To manage this space, AURORA-Q partitions it into M logical units (e.g., 256 subsets). Each unit is assigned a unique subset ID, ranging from 0 to $M - 1$, which represents a specific amplitude range. This granularity (e.g., 2 GB) is selected to balance metadata overhead and I/O efficiency in Section IV-E. AURORA-Q then distributes these subset IDs across nodes through logical ownership. Unlike GPU-centric simulators [5], [12], [13] that partition within physical GPU VRAM or DRAM capacity, AURORA-Q assigns each node a fixed logical subset ID range (e.g., $M/4$). For example, with 512 GB of amplitudes and $M = 256$, Node 0 receives subset IDs [0, 63] (128 GB). If this assigned range exceeds the node's total memory capacity (e.g., GPU VRAM and DRAM), AURORA-Q maps the overflow subsets to storage to be managed during runtime. At this stage, only the logical mapping is configured. The physical data corresponding to each subset ID is not yet generated.

Generating Target Subset. After the full state vector is logically partitioned across nodes by subset IDs, AURORA-Q uses static analysis for target subset generation. Target subset is a data segment that contains a specific range of amplitudes used for quantum gate operations within a task. Since the circuit structure is fixed, this analysis is performed at the task level (e.g., fused-gate unit group) to precompute access patterns, enabling sequential generation of required subsets at runtime. Specifically, AURORA-Q adopts the bitwise indexing mechanism from *ScaleQsim* [12] to identify the

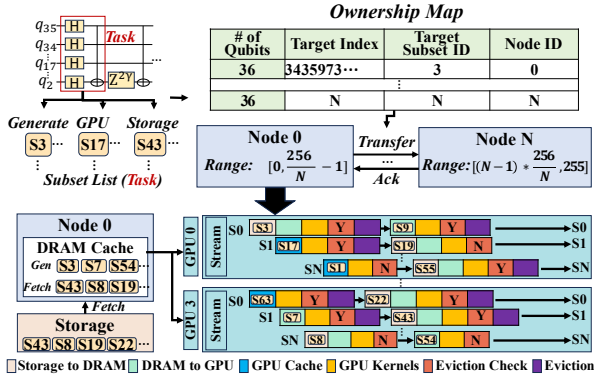


Fig. 6: Asynchronous execution pipeline of AURORA-Q. memory offsets that the task updates. To achieve efficient index generation, AURORA-Q derives memory offsets by varying only the bit positions of the affected qubits. This bitwise indexing allows the system to identify update regions with $O(K)$ complexity (e.g., where K is the number of affected qubits) without scanning the full state vector. For example, as shown in *Target Subset Generator* of Figure 5, AURORA-Q calculates the offset as $2^{34} + 2^{35}$ for the $|11\rangle$ state on qubits q_{34} and q_{35} (i.e., Task T49).

Once all target indices for a task are obtained, AURORA-Q performs a lookup against *Subset ID List* and generates target subset only if it has not been instantiated in a previous task. In GPU-centric simulators [12], [13], the full state vector is distributed statically in GPU VRAM, whereas target subset in AURORA-Q is instantiated on-demand per task across memory tiers, including storage. Thus, the subset ID serves as a lookup key to trigger physical generation and the necessary cache or I/O operations.

Decoupled Residency Management. As the full state vector is distributed across multiple nodes, AURORA-Q decouples global logical ownership from local physical residency when managing target subsets. To achieve this, AURORA-Q utilizes *Ownership Map* for the logical assignment of each target subset ID to a node ID. By managing only target indices, target subset IDs, and node IDs per task (i.e., fused-gate unit group) while excluding physical location information, *Ownership Map* provides a consistent global view of subset ownership without tracking residency transitions. Once the owner node is determined, the actual data location is managed independently at the node level. Each node determines the precise physical tier (i.e., GPU VRAM, DRAM, or Storage) where the data is resident using its local cache controller. This decoupled approach enables each node to independently manage frequent residency transitions for its assigned subset IDs, eliminating the need for global synchronization when subsets move between tiers. Thus, AURORA-Q enables handling full state vectors that exceed GPU VRAM and DRAM capacity by leveraging storage alongside fast but limited memory.

B. Asynchronous Execution Pipeline

To overcome the latency overhead of memory tiers, AURORA-Q employs an asynchronous execution pipeline.

Per-Task Coordination. As shown at the top of Figure 6, the

set of affected target indices varies for each quantum gate, requiring a unique combination of target subsets for every task. To handle this dynamic dependency, AURORA-Q performs per-task coordination. For a requested task (e.g., q_{35}, q_{17}), node 0 identifies required target subset IDs (e.g., S3, S17) and generates *Ownership Map*. This map logically maps subset IDs to owner nodes, abstracting physical residency. Precise locations (e.g., GPU VRAM, DRAM, or Storage) are resolved locally via each node’s cache controller, avoiding global synchronization during tiered memory transitions. Node 0 then broadcasts this map to all nodes. Upon receipt, each nodes reference the map to determine responsibility based on its assigned ID ranges. Although broadcast once per task, this CPU-side process overlaps with GPU execution of preceding tasks, keeping coordination overhead hidden and negligible (Section IV-D). *Ownership Map* size depends on target subsets per task rather than total state vector size (e.g., cumulative total of 0.63 seconds and 103.85 KB for 36 qubits, and 2.53 seconds and 1,142 KB for 40 qubits).

Overlapped Execution. Once coordination concludes, each node begins gate execution on target subsets within its assigned ID range. For example, with $N = 4$ nodes and $M = 256$ target subsets (Section III-A), node 0 is responsible for IDs [0, 63] and node 3 for [192, 255]. As shown at the bottom of Figure 6, the asynchronous pipeline is illustrated using node 0. Node 0 first identifies task-required target subsets within its subrange. By referencing the cache lists of local GPUs and DRAM to determine the current location, the node selects the appropriate preparation path: 1) directly accessing target subsets already present in GPU cache via cache lookup and using them for computation, 2) retrieving target subsets (i.e., S43, S8) from storage to DRAM and transferring them to GPUs for computation, or 3) generating new target subsets (i.e., S3, S7) for first-time access and placing them into *DRAM Cache Pool*.

After fetching, each GPU utilizes multiple CUDA streams (e.g., S0, S1) to process target subsets. A key AURORA-Q design feature is the integration of data management with computation. As depicted in the legend, each stream includes cache management (i.e., *Eviction Check*, *Eviction*) alongside data movement and computation. This design decouples I/O-bound and cache management from compute-bound execution. For example, while Stream S1 on GPU 3 waits for a high-latency fetch of S43, GPU can execute computation on Stream S0 (i.e., S63) and perform an eviction check (i.e., S22 on Stream S0). To prevent I/O contention from multiple concurrent fetches, AURORA-Q serializes high-latency storage accesses through sequential queuing, ensuring efficient utilization of the storage bandwidth of the underlying storage system.

Cross-Resource Gate Execution. Our execution strategy applies consistently across both inter- and intra-node cases when a task spans multiple resources. Specifically, when a task spans multiple nodes, AURORA-Q eliminates runtime location discovery overhead by utilizing the received *Ownership Map*. While this map statically identifies the owner node, the

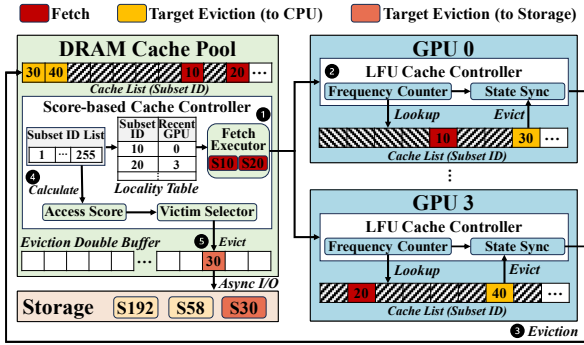


Fig. 7: Two-level cache management of AURORA-Q.

data’s actual physical location may change dynamically due to concurrent operations (e.g., cache eviction). Therefore, upon receiving a request, the owner node performs a local lookup in its cache lists to resolve the precise memory tier in real-time. This mechanism isolates dynamic data tracking within the owner node, ensuring that *Ownership Map* remains valid for request routing without requiring global updates during gate execution. Based on this routing, each node updates only its assigned target subsets, ensuring exclusive access and preventing conflicts.

When the task spans multiple GPUs within a single node, AURORA-Q utilizes CUDA Peer-to-Peer access. To ensure correctness, AURORA-Q assigns exclusive update responsibility to each GPU, treating remote data accessed via CUDA P2P as read-only operands and restricting writes to local memory. This strategy is feasible because applying a quantum gate is equivalent to a matrix–vector multiplication, where each output element is computed as a linear combination of input elements and does not depend on other simultaneous updates. Thus, this process inherently avoids race conditions by preventing write conflicts, eliminating the need for synchronization within the gate kernel.

C. Two-level Cache Hierarchy

To maximize asynchronous pipeline efficiency and reduce I/O latency, AURORA-Q uses a two-level cache: GPU VRAM with LFU for frequent access and DRAM with a score-based policy for improved hit rates.

GPU-resident Cache. AURORA-Q configures each GPU to store a small number of frequently accessed target subsets to exploit inherent access skew (as described in Section II-A). This cache serves as the highest-bandwidth memory region, providing immediate access to target subsets required for the gate kernel. As shown in Figure 7, AURORA-Q assigns a dedicated *LFU Cache Controller* to each GPU. When *Fetch Executor* in *DRAM Cache Pool* transfers required subsets (e.g., S10) to GPU (①), this controller immediately updates *Frequency Counter* in the cache list to prioritize high-reuse data (②), allowing the controller to track the reuse frequency. Additionally, the controller periodically reduces access counters via an aging mechanism to prevent cache pollution and adapt to changes in data locality.

To ensure consistency between GPU VRAM and DRAM without global synchronization, GPU-resident cache enforces a fine-grained, stream-level *State Sync* on target subset

selected for eviction. If the cache becomes full, the controller checks the counters to ensure that target subset being evicted has completed all pending gate kernels on its assigned stream and reflects the most recent data state. This fine-grained synchronization allows other streams to continue execution without global blocking. After the synchronized state is confirmed, any frequency ties (e.g., multiple target subsets with the identical minimum access count) are resolved by selecting the oldest candidate. The controller then evicts the least-frequently used target subset to *DRAM Cache Pool* using a device-to-host transfer (i.e., S30, S40) (③).

Host-side DRAM Cache. As shown in Figure 7, each node maintains *DRAM Cache Pool* that serves as an intermediate tier between GPU VRAM and storage. DRAM cache provides a larger capacity than GPU, allowing it to hold more target subsets, but its capacity remains limited, so only a subset of all target subsets can stay resident. *DRAM Cache Pool* is managed by a dedicated *Score-based Cache Controller*. It maintains a cache list that tracks all target subsets stored in DRAM, along with metadata (e.g., access count, last-used time, a consistency state to prevent duplicate target subset placement across memory tiers). The controller also maintains *Locality Table* that records the recent GPU ID for each target subset to exploit execution locality within the node. When a task requests target subset, the controller checks the cache list. If the subset is resident, the controller references *Locality Table*, identifies the previously assigned GPU, and transfers the data directly via *Fetch Executor* (①). If target subset is not resident, *DRAM Cache Pool* loads it from storage and forwards it to GPU, ensuring that GPU receives the required data for execution.

To manage DRAM-level cache efficiency, *Score-based Cache Controller* applies a hybrid scoring mechanism to select a DRAM-level eviction victim. For each resident target subset, the controller computes a reuse score based on access recency and frequency, $reuse_score = \frac{access_count}{\Delta t + \epsilon}$, where Δt is the time since the last access and ϵ is a small constant for numerical stability (④). Before eviction, the controller verifies the consistency state to prevent target subset from being redundantly placed across memory tiers when it is relocated to storage. Then, the victim target subset is forwarded to *Eviction double buffer*, which asynchronously offloads it to storage (⑤). The double-buffering mechanism overlaps the eviction and storage I/O with computation to avoid execution stalls.

D. Adaptive Resource Control

During execution, fixed parameters for the memory hierarchy and kernel configuration can cause inefficiencies when they do not match task characteristics. A static cache size may trigger unnecessary I/O or OOM failures, and a fixed block count can underutilize threads for small tasks or increase memory pressure for large ones. To address these, AURORA-Q adjusts cache allocation, GPU assignment, and kernel parameters based on the task and hardware state.

Memory Hierarchy Provisioning. Algorithm 1 shows the adaptive parameter calculation for AURORA-Q’s

Algorithm 1 Memory Hierarchy Provisioning.

```
1: Input qubit count  $Q$ , # of GPUs  $G$ , # of subsets  $M$ 
2: Input  $\text{SAFE\_CAP\_GPU} = 0.8$ ,  $\text{SAFE\_CAP\_DRAM} = 0.8$ 
3: Output GPU cache size, DRAM cache size, # of streams per GPU
4: compute total state size  $T = 2^Q \times \text{sizeof}(\text{complex})$ 
5: compute subset size  $S = T/M$ 
6: measure available GPU VRAM on a sample device
7:  $\text{MAX\_CACHE\_GPU} = (\text{SAFE\_CAP\_GPU} \times \text{GPU\_mem})/S$ 
8:  $\text{MAX\_STREAM} = \min((\text{GPU\_mem}/S), 4)$ 
9: allocate per-stream GPU buffers of size  $S$  for all  $G$  GPUs
10: create  $\text{MAX\_STREAM}$  CUDA streams on each GPU
11:  $\text{MAX\_CACHE\_DRAM} = (\text{SAFE\_CAP\_DRAM} \times \text{DRAM\_mem})/S$ 
12: return ( $\text{MAX\_CACHE\_GPU}$ ,  $\text{MAX\_CACHE\_DRAM}$ ,  $\text{MAX\_STREAM}$ )
```

hierarchical architecture, determining GPU cache size (MAX_CACHE_GPU), CPU cache size (MAX_CACHE_DRAM), target subset size (S), and the number of streams per GPU (MAX_STREAM). Input parameters include the number of qubits (Q), GPUs (G), target subsets (M), and safety capacity ratios (set to 0.8 for both GPU VRAM and DRAM). AURORA-Q first derives the total state size (T) from Q (Line 4), and computes the subset size $S = T/M$ (Line 5) (e.g., for $Q = 36$, T is 512 GB. If $M = 256$, then S is 2 GB). AURORA-Q profiles the available GPU VRAM (Line 6) and calculates MAX_CACHE_GPU . Assuming an 80 GB GPU, MAX_CACHE_GPU is set to 32 (calculated as $0.8 \times 80\text{GB}/2\text{GB}$) (Line 7). This safety margin ensures the cache does not consume space required for compute kernels.

Similarly, MAX_STREAM is tuned to the minimum of ($\text{GPU VRAM} / S$) and an empirical threshold to minimize overhead (e.g., set to 4 as demonstrated in Section IV-E) (Line 8). For an 80 GB GPU, this results in $\min(40, 4)$, capping MAX_STREAM at 4. Once set, AURORA-Q allocates per-stream buffers and creates CUDA streams (Lines 9-10). Finally, AURORA-Q sets the host-side MAX_CACHE_DRAM using DRAM safety ratio (e.g., with 256 GB DRAM, $0.8 \times 256\text{GB}/2\text{GB} \approx 102$ subsets) (Line 11). This process balances I/O concurrency and computational resources without manual tuning.

Context-aware Runtime Control. Algorithm 2 orchestrates the adaptive process AURORA-Q applies to each target subset (s) owned by the current node (Line 3). This process dynamically assigns each subset to an appropriate GPU and executes it with safe kernel parameters.

AURORA-Q first performs adaptive GPU assignment by referencing the node-local *Locality Table*, which tracks execution history within the node to preserve locality (Line 4). The primary rule is to minimize data movement across GPUs. If s has a previous GPU assignment recorded in *Locality Table*, it is reassigned to the same GPU (Lines 5-6). This strategy increases the likelihood of a GPU cache hit and avoids unnecessary cache interference. AURORA-Q ensures that s competes for cache space only on its previously assigned GPU. If s has no prior location (i.e., first access), a cache-aware policy selects the GPU with the lowest LFU score and updates *Locality Table* (Lines 7-9). Once assigned, a CUDA stream is selected using modulo arithmetic against MAX_STREAM , and a stream-safe buffer is used to prevent data corruption during concurrent execution (Lines 11-12). AURORA-Q then maintains cache coherence. If s is not resident in the GPU cache, an LFU victim is evicted, and s is fetched from DRAM

Algorithm 2 Context-aware Runtime Control.

```
1: Input Target subset list  $S$ , # of GPUs  $G$ , qubit count  $Q$ 
2: Output GPU assignment, residency handling, kernel launch parameters
3: for each subset  $s \in S$  owned by the current node do
4:   Check  $s$  in Locality table ▷ Retrieve locality info
5:   if Locality table records a previous GPU then
6:      $\text{gpu} \leftarrow$  recorded GPU ID ▷ Preserve locality
7:   else
8:      $\text{gpu} \leftarrow$  GPU with the lowest LFU score ▷ Cache-aware selection
9:     update Locality table with ( $s \rightarrow \text{gpu}$ )
10:  end if
11:   $\text{stream} \leftarrow \text{smod } \text{MAX\_STREAM}$ 
12:  use stream-safe buffer associated with ( $\text{gpu}, \text{stream}$ )
13:  if  $s$  is not resident in GPU VRAM then ▷ Check current location
14:    evict LFU victim if GPU cache is full
15:    fetch subset  $s$  from DRAM/Storage into ( $\text{gpu}, \text{stream}$ )
16:  else
17:    update LFU counter (frequency counter) for  $s$ 
18:  end if
19:  derive number of target qubits  $k_{\text{gate}}$  and split threshold  $k \leftarrow 5 + k_{\text{gate}}$ 
20:  compute baseline blocks based on remaining parallelism ( $2^{Q-k}$ )
21:  if thresholds exceeded then
22:    activate safe_mode and adjust blocks
23:  end if
24:  launch kernel for  $s$  on ( $\text{gpu}, \text{stream}$ ) with finalized (blocks, threads)
25: end for
```

or storage (Lines 13-15). If already resident, *frequency counter* is updated to record the hit (Lines 16-17).

Finally, AURORA-Q applies adaptive kernel configuration to guarantee execution stability. It derives the task complexity by setting the split threshold $k = 5 + k_{\text{gate}}$ following *Qsim*'s policy [14] and computes a baseline block count from the remaining parallelism (2^{Q-k}) (Lines 19-20). AURORA-Q then checks if these parameters exceed GPU VRAM limits. If the configuration is not feasible, AURORA-Q activates *safe_mode* and dynamically throttles the block count (Lines 21-22). This reduces parallelism when a task demands more memory than GPU can provide (e.g., limiting a 42-qubit task to 2^{27} blocks (≈ 64 GB) on an 80 GB GPU). After the parameters are finalized, the kernel is launched with the safe (*blocks*, *threads*) setting (Line 24), ensuring that execution remains within GPU VRAM limits.

E. AURORA-Q Implementation

We implement AURORA-Q by extending the Google *Qsim* [14]. While *Qsim* is limited to single GPU execution, we redesign its memory management and execution model to exploit the tiered-memory hierarchy, modifying roughly 1500 lines. Specifically, AURORA-Q integrates a tiered-memory architecture with asynchronous execution, two-level cache, and adaptive resource management to enable dynamic state partitioning, I/O-computation overlap, and hardware-aware parameter tuning.

IV. EVALUATION

A. Evaluation Setup

Hardware Specification. We evaluate AURORA-Q on Perlmutter, which is ranked within the top 20 of the TOP500 list. The system consists of 3,072 CPU nodes and 1,792 GPU nodes, and our experiments use only the GPU nodes. Each GPU node is equipped with an AMD EPYC 7764 processor, 256 GB of DDR4 DRAM, and four NVIDIA A100 GPUs connected via PCIe 4.0. Each A100 GPU provides 80 GB of HBM2e memory, and the GPUs within a node are interconnected through NVLink. Storage is provided by the

TABLE II: Our benchmark circuits and their size (# of gates).

Circuit	Complexity	Number of qubits						
		36	37	38	39	40	41	42
qft	High	638	667	696	725	754	783	812
qv	Medium	432	444	456	468	480	492	504
vqc	High	1140	1164	1188	1212	1236	1260	1284
qsvm	Medium	71	73	75	77	79	81	83
random	Medium	540	550	570	580	600	610	630
ghz	Low	36	37	38	39	40	41	42
vqe	High	4280	4400	4520	4640	4760	4880	5000

system’s Lustre parallel file system, with a quota of 20 TB.

Benchmark. We evaluate AURORA-Q using the Quantum Fourier Transform (qft) circuit, which is characterized by dense entanglement and high gate complexity. In addition to qft, six other representative circuits are included in our benchmark suite. The seven circuit types used in our evaluation are summarized in Table II.

Baselines. We compare AURORA-Q against state-of-the-art GPU-centric simulators: *cusvaer* [13] (IBM Qiskit [19] using cuStateVec), *ScaleQsim* [12], *HyQuas* [6], and *Atlas* [5]. While *Atlas* and *cusvaer* support optional DRAM offloading, we compare against their GPU-centric modes. Since DRAM offloading incurs PCIe overhead, the GPU-centric mode reflects peak performance. As AURORA-Q outperforms these baselines in their fastest GPU-centric modes (Section IV-B, IV-C), comparison with slower offloading modes is unnecessary. In addition, these simulators lack storage integration. Even with DRAM offloading, they cannot scale to 43 qubits under an identical configuration. Scaling to 43 qubits requires storage beyond DRAM capacity, which AURORA-Q supports.

Exclusion of Storage-extended Simulators. Although we intended to compare AURORA-Q with storage-extended simulators, *BMQSim* [10], and *SnuQS* [9], they are excluded from direct comparison because their source codes are not publicly available. Their reported performance and scalability also differ from GPU-centric simulators. *BMQSim* supports only up to 33 qubits (e.g., theoretically up to 42 qubits, but validated only to this scale), and *SnuQS*, while supporting 42 qubits, shows execution times from minutes to hours (e.g., 41.4 minutes for 39 qubits and 16.72 hours for 42 qubits), whereas AURORA-Q achieves execution speeds comparable to GPU-centric simulators while retaining storage-extended scalability. Given this disparity, we instead validate the storage efficiency of AURORA-Q through compression analysis in Section IV-E.

Feasibility. A 43-qubit simulation requires approximately 64 TB of memory, requiring at least 128 nodes (512 GPUs) in AURORA-Q. With 128 nodes, the combined GPU VRAM and DRAM capacity is insufficient to store the full state vector after applying the safety cap in Algorithm 1 (e.g., 32 TB of VRAM and 25.6 TB of DRAM). The remaining 6.4 TB is placed on the Lustre parallel file system, which provides sufficient capacity for offloaded data. Thus, AURORA-Q supports 43-qubit simulation on 128 nodes, while GPU-centric simulators remain non-executable.

B. Performance with SOTA

of Nodes. Figure 8 compares the performance of AURORA-Q and GPU-centric simulators (*ScaleQsim* and *cusvaer*) across node scales on qft. AURORA-Q shows scalable performance as node count increases. At 35 qubits (1 node), AURORA-Q com-

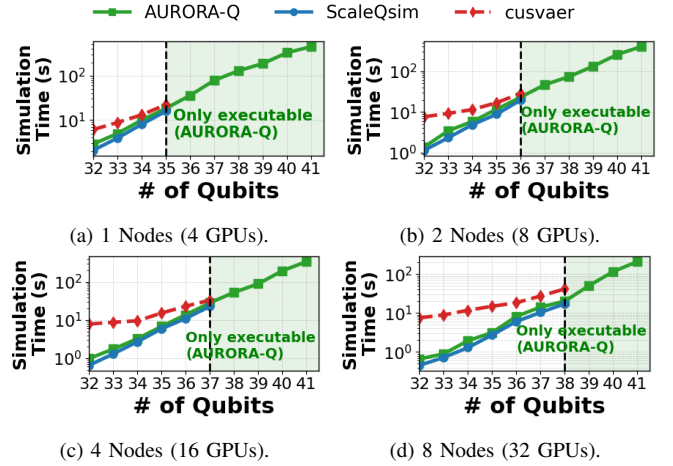


Fig. 8: Performance comparison between AURORA-Q, *ScaleQsim* and *cusvaer* across # of nodes on qft circuits (log-scale).

pletes in 18.29s, close to *ScaleQsim* (15.83s) and 1.22 $\times$ faster than *cusvaer* (22.24s). This marks the execution boundary for GPU-centric simulators on a single node, while AURORA-Q continues beyond this limit, completing 36 qubits in 36.07s and scaling to 41 qubits without increasing node count. This behavior persists at larger scales. At 38 qubits (8 nodes), AURORA-Q takes 20.19s, close to *ScaleQsim* (17.16s) and faster than *cusvaer* (41.61s). Both GPU-centric simulators fail beyond 38 qubits due to memory limits, whereas AURORA-Q completes 40 qubits in 116.10s. Thus, AURORA-Q is the only simulator that executes 41-qubit simulations across all node configurations.

These results highlight the efficiency of AURORA-Q’s decoupled design. *ScaleQsim* achieves high throughput through static partitioning but fails once the state vector exceeds GPU capacity. AURORA-Q achieves competitive performance by adapting *ScaleQsim*’s bitwise indexing mechanism, while hiding residency management overhead through its asynchronous execution pipeline. *cusvaer* faces scalability limits due to frequent global synchronization for cross-node gates in its distributed memory model. In contrast, AURORA-Q eliminates this bottleneck by decoupling global logical ownership from local physical residency via *Ownership Map*. Although coordination occurs per task, the overhead is masked by host-device overlapping, allowing nodes to resolve and execute tasks independently.

Various Circuits. Figure 9 compares simulation performance of AURORA-Q and GPU-centric simulators (*ScaleQsim* and *cusvaer*) across six circuits on 128 GPUs (32 nodes). As depicted, *ScaleQsim* and *cusvaer* fail beyond 40 qubits due to GPU memory limits, while AURORA-Q scales to 42 qubits for all circuits using its tiered-memory design. At 38–40 qubits, where all simulators operate, AURORA-Q shows performance comparable to *ScaleQsim* and outperforms *cusvaer*. For example, for the 40-qubit random circuit, AURORA-Q completes in 13.21s, compared to 5.66s for *ScaleQsim*. The higher latency relative to *ScaleQsim* incurs overhead from dynamic subset generation, host-to-device fetching, and stream scheduling. However, AURORA-Q mitigates this cost through computation

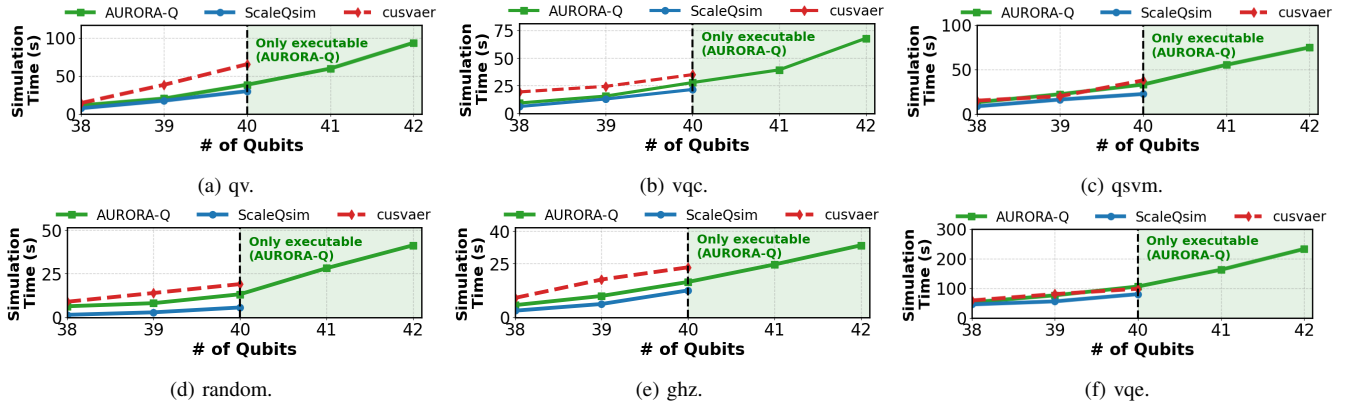


Fig. 9: Performance comparison for AURORA-Q, *ScaleQsim*, and *cusvaer* across diverse circuits using 128 GPUs (32 Nodes).

overlap. Thus, AURORA-Q achieves a $1.43\times$ speedup over *cusvaer* (19.02s) due to minimized global coordination.

Notably, AURORA-Q demonstrates robust scalability beyond GPU-centric simulators. At 41-42 qubits, AURORA-Q is the only simulator that completes all circuit types. By dynamically orchestrating target subsets across GPU VRAM, DRAM, and storage, AURORA-Q overcomes the memory wall that restricts SOTA simulators and enables simulations at unattainable scales.

C. Scalability with SOTA

Weak Scalability. Figure 10 compares the weak scalability of AURORA-Q with SOTA simulators (*ScaleQsim*, *cusvaer*, *HyQuas*, *Atlas*) using a qft circuit. We varied the number of qubits and GPUs simultaneously (e.g., from 28 qubits on 1 GPU to 36 qubits on 256 GPUs) to evaluate scalability as problem size and resources increase. As depicted, AURORA-Q takes 0.47s at 28 qubits and 7.58s at 36 qubits. At 36 qubits, AURORA-Q achieves $2.91\times$ and $1.23\times$ speedup over *cusvaer* and *HyQuas*, respectively. Although *ScaleQsim* and *Atlas* show faster simulation time at this scale, AURORA-Q demonstrates scalability trends competitive to these GPU-centric simulators.

These results highlight the architectural efficiency of AURORA-Q. *ScaleQsim* achieves high throughput through static partitioning but fails once the state vector exceeds GPU capacity. AURORA-Q achieves competitive performance by adapting *ScaleQsim*'s bitwise indexing mechanism, while hiding the overhead of dynamic stream scheduling and subset generation through the asynchronous pipeline. *cusvaer* faces scalability limits due to frequent global synchronization for cross-node gates in its distributed memory model. In contrast, AURORA-Q uses *Ownership Map* only for node-level mapping and resolves precise residency locally at each node. Its asynchronous pipeline masks local metadata lookups and tiered-memory transfers, keeping coordination cost negligible as node count and problem size increase.

Strong Scalability. Figure 11 compares the strong scalability of AURORA-Q with SOTA simulators as the number of GPUs increases from 256 to 512. As depicted, AURORA-Q outperforms *cusvaer* while maintaining scalability. At 256 GPUs, AURORA-Q takes 10.21s at 38 qubits, achieving a $2.70\times$ speedup over *cusvaer* (27.53s). At 512 GPUs, simulation time at 42 qubits is 96.42s for AURORA-Q and 112.82s for *cusvaer*,

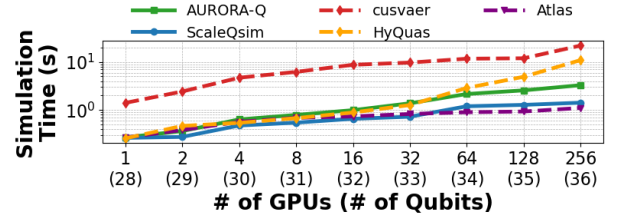


Fig. 10: Weak scalability of AURORA-Q and SOTA with GPU scales on qft circuits (log-scale).

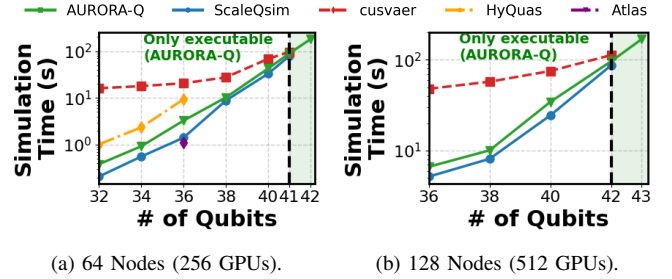


Fig. 11: Strong scalability comparison across different # of GPUs (log-scale).

resulting in a $1.17\times$ speedup. In high-resource scales, where the state vector is fully distributed across GPUs, AURORA-Q shows a performance trade-off compared to GPU-centric simulators. For example, at 256 GPUs (40 qubits), AURORA-Q takes 42.48s, slower than *ScaleQsim* (33.02s). This difference arises because AURORA-Q's unified tiered memory architecture maintains consistent CPU-side subset generation and per-subset management regardless of VRAM capacity, incurring stream scheduling overhead even when the state vector could theoretically fit in GPU VRAM. However, this design enables AURORA-Q to complete 42-qubit on 256 GPUs and extend to 43 qubits on 512 GPUs, a scale where *HyQuas* and *Atlas* fail beyond 36 qubits. Notably, although we attempted to generate static execution plans for *Atlas* in these configurations, it failed to produce a valid execution plan or timed out, revealing the inflexibility of static planning. Furthermore, *ScaleQsim* and *cusvaer* cannot run due to memory limits. Thus, AURORA-Q supports stable execution at these scales, with the 43-qubit limit determined by the available 20TB PFS capacity (as described in Section IV-A).

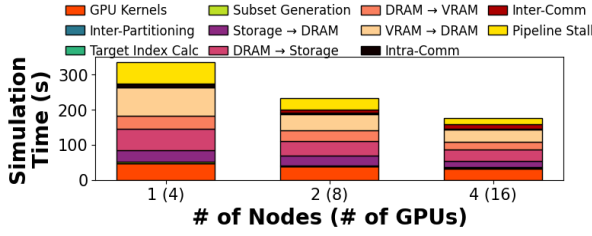


Fig. 12: Time analysis of AURORA-Q.

D. Time Analysis

Since AURORA-Q relies on a tiered memory hierarchy where the state vector is distributed between GPU VRAM, DRAM, and storage, minimizing I/O latency is crucial. Thus, it is necessary to analyze how scaling affects both data movement and computation costs. Figure 12 shows the simulation time breakdown for a 40-qubit simulation across 1 to 4 nodes.

Initialization and Planning. *Inter-partitioning* incurs minimal overhead, increasing from 0.05s (1 Node) to 0.08s (4 Nodes). *Subset Generation* time also decreases with scaling: 3.48s (1 Node), 2.42s (2 Nodes), and 1.93s (4 Nodes), as the distributed state vector reduces the number of target subsets generated per node. In contrast, *Target index Calc* time increases from 0.43s to 0.61s and 0.94s because mapping target indices to distributed locations becomes more complex. However, these costs remain negligible, accounting for less than 1.7% of the total simulation time.

Data Movement. Data movement dominates runtime in 1 Node, but decreases substantially with scaling. *Storage to DRAM* read time decreases from 32.66s to 18.74s, and *DRAM to Storage* flush time drops from 62.45s to 32.22s. *VRAM to DRAM* eviction also decreases from 82.34s to 34.88s, indicating that distributing the state vector reduces memory pressure and lowers eviction and flushing frequency. As each node stores a smaller portion of the full state vector, transfers involve smaller data volumes, reducing overall I/O load. *DRAM to VRAM* transfer time similarly decreases from 35.84s to 21.27s as each node handles less data.

Computation and Communication. *GPU Kernels* decreases from 47.20s to 31.62s, indicating that AURORA-Q maintains execution efficiency as the workload per GPU decreases. *Intra-comm* decreases from 10.50s to 4.22s because reduced local data volume lowers synchronization cost. In contrast, *Inter-comm* increases from 0.00s (1 Node) to 6.89s (4 Nodes) as distributed coordination becomes necessary due to *Ownership Map* broadcasts across nodes. However, while absolute communication time increases with scale, this overhead is masked by the asynchronous pipeline. As coordination is processed on the host CPU and overlaps with GPU execution, it does not block computation. Thus, *Inter-comm* cost (+6.89s) is outweighed by the reduction in *Pipeline Stall* (-43.59s), which decreases from 59.96s to 16.37s. These results show that AURORA-Q achieves scalability by hiding latency and reducing I/O-related stalls.

E. Sensitivity Analysis

Impact of Compression. State vector compression is a strategy to overcome memory capacity limits. To evaluate its

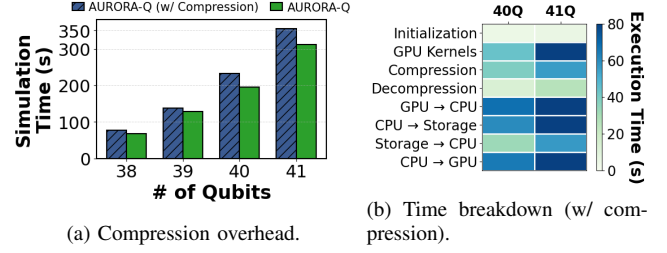


Fig. 13: Compression overhead in AURORA-Q.

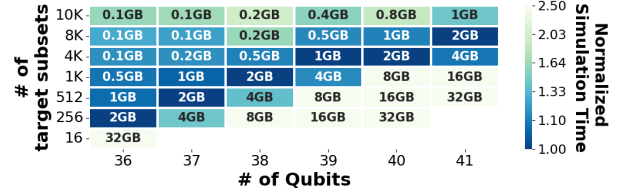


Fig. 14: Impact of granularity in AURORA-Q.

efficacy, we implement a Zstandard (zstd) lossless compression module and compare simulation time against AURORA-Q (w/o compression) on 4 nodes (16 GPUs), as shown in Figure 13a. Contrary to expectations, AURORA-Q (w/o compression) performs better. Enabling compression increases total simulation time by $1.08\times$ – $1.13\times$. The breakdown in Figure 13b shows that CPU-side compression and decompression outweigh reduced storage I/O. This overhead is most visible in the fetch path, where decompression adds a serialization step before CPU→GPU transfer and increases GPU idle time. Since AURORA-Q’s asynchronous pipeline hides uncompressed I/O latency, the added CPU work becomes a bottleneck.

While uncompressed execution shows the best performance, compression becomes necessary when capacity limits are reached. In such cases, our design prioritizes scalability rather than raw compression throughput. Although GPU-side compression (*BMQSim* [10]) offers faster processing, it couples auxiliary work with gate execution on the GPU, causing resource contention at scale. Thus, if compression is required, AURORA-Q performs it on the host to keep GPU resources dedicated to gate operations and maintain throughput stability.

Impact of Granularity. Figure 14 presents a sensitivity analysis of granularity for qubit counts from 36 to 41 on a single node. We vary target subset count from 16 to 10K to evaluate the impact of data transfer granularity on execution efficiency. As depicted, AURORA-Q consistently achieves the best performance when the per-subset size falls within the 1-4 GB range, regardless of qubit count. For example, at 41 qubits (16 TB), the best simulation time of 436.26s is achieved with 8,192 subsets (≈ 2 GB), providing a $2.50\times$ speedup over the coarse-grained 1,024 subsets (≈ 16 GB, 1092.15s) and competitive performance to 4,096 subsets (≈ 4 GB, 471.08s). Similarly, at 36 qubits (512 GB), the 2 GB configuration (256 subsets) delivers a $5.52\times$ speedup over the 32 GB case (16 subsets, 199.14s). This confirms that the 1-4 GB range provides the most efficient for our asynchronous pipeline. However, performance degrades outside this range. Target subsets larger than 4 GB induce pipeline stalls as transfer latency exceeds computation time, while those smaller than

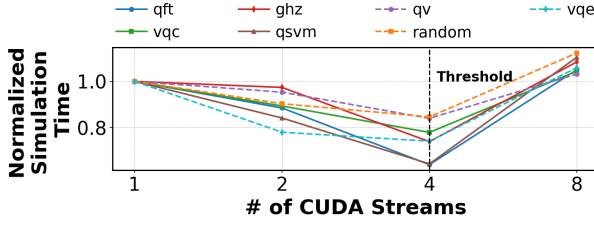


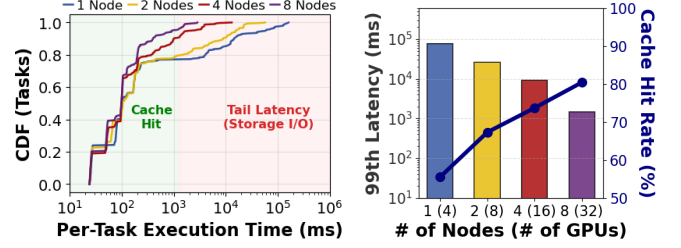
Fig. 15: Impact of CUDA streams of AURORA-Q.

1 GB incur higher metadata lookup overhead and inefficient PCIe transfer.

Impact of Stream. Figure 15 shows the impact of CUDA stream counts on simulation performance. Other parameters, such as target subset size (e.g., 2 GB) and number of GPUs (e.g., 1 Node–4 GPUs), are fixed to isolate the stream effect. While a single stream executes sequentially, increasing the stream count to 4 yields the best performance, achieving $1.19\times$ – $1.57\times$ speedups across all circuits. This confirms that AURORA-Q’s asynchronous pipeline hides I/O and cache management latencies by overlapping them with computation. However, when the stream count increases to 8, performance degrades compared to 4 streams and, for some circuits (e.g., qft, vqc, qsvm), even falls below the 1-stream baseline. For example, qft requires 450.24 s at 8 streams, worse than the baseline. This occurs because context-switch overhead and GPU contention outweigh pipeline benefits. Thus, excessive streams increase scheduling cost and fragment memory access, confirming that the design in Section III-D, which limits the stream count (e.g., capped at 4 or determined by memory capacity), is necessary for stable performance.

Impact of Cache Efficiency. Figure 16 shows the impact of scalability on performance stability and cache efficiency. We evaluated a 40-qubit simulation from 1 to 8 nodes (4–32 GPUs) to examine how additional cache capacity mitigates I/O latency. As shown in Figure 16a, increasing the node count shifts the latency distribution left and reaches a CDF of 1.0, as larger cache capacity reduces pipeline stalls and allows execution to proceed without waiting for fetch operations. The long tail at small scales (e.g., 1 node) arises from tasks requiring multiple target subsets, where execution completes only after non-resident subsets are fetched from storage. This behavior aligns with Figure 16b, where reductions in P99 latency mirror the CDF trend. Even the single-node case achieves a 56.1% hit rate due to AURORA-Q’s two-level hierarchy and *Locality Table*, which utilizes LFU and score-based policies to exploit execution locality for frequently accessed subsets. Scaling to 8 nodes increases the hit rate to 79.3% as expanded cache capacity, and *Locality Table*-driven local data reuse reduces P99 latency from ~ 80 s to ~ 1.5 s.

Fidelity. To demonstrate the fidelity of AURORA-Q, we evaluated simulation fidelity on 4 nodes (16 GPUs) using *Qsim* as the baseline. The comparison includes results up to 32 qubits, the maximum scale supported by *Qsim*, as shown in Table III. Results show that AURORA-Q maintains consistent fidelity across the evaluated range. For qft, ghz, and qsvm, fidelity reaches 1.0, while random and vqe show small deviations above $1 - 10^{-6}$. These deviations reflect minor numerical



(a) CDF of per-task execution time. (b) Latency and cache hit rate.

Fig. 16: Impact of scalability on performance stability and cache efficiency in AURORA-Q.

TABLE III: Measured fidelity (28–32 qubits) between *Qsim* and AURORA-Q across various circuits.

Circuit	28 Qubit	29 Qubit	30 Qubit	31 Qubit	32 Qubit
qft	1.00000000	1.00000000	1.00000000	1.00000000	1.00000000
qv	$1 - 10^{-9}$	1.00000000	$1 - 10^{-10}$	$1 - 10^{-8}$	1.00000000
vqc	$1 - 10^{-8}$	$1 - 10^{-9}$	1.00000000	$1 - 10^{-9}$	$1 - 10^{-9}$
qsvm	1.00000000	1.00000000	1.00000000	1.00000000	1.00000000
random	$1 - 10^{-7}$	$1 - 10^{-6}$	$1 - 10^{-7}$	$1 - 10^{-6}$	$1 - 10^{-6}$
ghz	1.00000000	1.00000000	1.00000000	1.00000000	1.00000000
vqe	$1 - 10^{-7}$	$1 - 10^{-7}$	$1 - 10^{-6}$	$1 - 10^{-9}$	$1 - 10^{-6}$

drift rather than tiered-memory execution. The absence of increasing error trends with qubit count indicates that the verified fidelity up to 32 qubits is preserved beyond this scale.

V. RELATED WORKS

Quantum Simulation Beyond Memory Limits. To maximize scalability, many studies have optimized large-scale quantum simulation through advanced memory management schemes. Prior work focuses on extending state vector capacity via circuit partitioning, distributed execution, and hybrid memory management [5], [6], [12]. Other studies improve memory utilization by optimizing storage offloading, data compression, and computation–I/O overlap [9], [10], [16]. In addition, several approaches combine full state vector simulation with amplitude sampling or sparsity-aware techniques to reduce computational complexity [20]–[22]. Our study aligns with prior work on scalable quantum circuit simulation. AURORA-Q introduces a unified tiered-memory architecture that overcomes memory and I/O limits through dynamic residency management. By tracking and adaptively distributing the state vector across memory tiers, AURORA-Q enables execution beyond physical memory capacity and supports larger qubit counts than SOTA simulators.

Memory and I/O Optimization in HPC Systems. Modern HPC systems incorporate memory and I/O optimization techniques to support data-intensive workloads (e.g., AMReX [23], Quantum ESPRESSO [24]). Previous studies [25], [26] extend effective memory capacity by distributing data across hierarchical memory tiers. Other studies [27], [28] overlap computation with communication or I/O to reduce pipeline stalls and improve throughput. Additionally, some studies [29], [30] explore locality-aware scheduling and multi-level cache management to reduce redundant transfers and mitigate high-latency bottlenecks. These approaches highlight techniques for improving scalability and efficiency in memory-intensive HPC workloads. Similarly, AURORA-Q faces comparable challenges in full state vector simulation, where the exponential state

vector spans multiple memory tiers and shifts the bottleneck to data movement. To address this, AURORA-Q employs an asynchronous pipeline that overlaps computation with I/O transfers and cache management. This enables AURORA-Q to execute memory-intensive workloads efficiently, mitigating I/O costs and sustaining throughput.

VI. CONCLUSION

In this paper, we design and implement AURORA-Q, a scalable quantum circuit simulator based on a unified tiered-memory architecture. By dynamically managing data residency across memory tiers, we optimize execution to overlap computation with data movement and cache management. We evaluate AURORA-Q on diverse quantum circuit benchmarks and show that it scales to 43 qubits on 512 GPUs, surpassing the limits of SOTA simulators, and achieves up to $2.91\times$ speedup. This is achieved by hiding I/O latency with up to a 79.3% cache hit rate and reducing pipeline stalls by 73%.

VII. ACKNOWLEDGMENT

This work was supported by the National Research Foundation of Korea (NRF) under Grant Nos. RS-2025-16070038, RS-2025-00554650, and RS-2023-00283799. This work was also supported by the Office of Advanced Scientific Computing Research, Office of Science, of the U.S. Department of Energy under Contract No. DE-AC02-05CH11231, and used resources of the National Energy Research Scientific Computing Center (NERSC). We acknowledge the use of LLMs to refine the academic language.

REFERENCES

- [1] J. Miguel-Ramiro, Z. Shi, L. Dellantonio, A. Chan, C. A. Muschik, and W. Dür, “Enhancing quantum computation via superposition of quantum gates,” *Physical Review A*, vol. 108, no. 6, p. 062604, 2023.
- [2] M. J. Renner and Č. Brukner, “Computational advantage from a quantum superposition of qubit gate orders,” *Physical Review Letters*, vol. 128, no. 23, p. 230503, 2022.
- [3] Y. Suzuki, S. Endo, K. Fujii, and Y. Tokunaga, “Quantum error mitigation as a universal error reduction technique: Applications from the nisy to the fault-tolerant quantum computing eras,” *PRX Quantum*, vol. 3, no. 1, p. 010345, 2022.
- [4] J. Preskill, “Quantum computing in the nisy era and beyond,” *Bulletin of the American Physical Society*, vol. 64, no. 9, 2019.
- [5] M. Xu, S. Cao, X. Miao, U. A. Acar, and Z. Jia, “Atlas: Hierarchical partitioning for quantum circuit simulation on gpus,” in *SC24: Int’l Conf. for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2024, pp. 1–17.
- [6] C. Zhang, Z. Song, H. Wang, K. Rong, and J. Zhai, “Hyquas: hybrid partitioner based quantum circuit simulation system on gpu,” in *Proceedings of the 35th ACM Int’l Conf. on Supercomputing*, 2021, pp. 443–454.
- [7] R. Fu, Z. Su, H.-S. Zhong, X. Zhao, J. Zhang, F. Pan, P. Zhang, X. Zhao, M.-C. Chen, C.-Y. Lu *et al.*, “Achieving energetic superiority through system-level quantum circuit simulation,” *arXiv preprint arXiv:2407.00769*, 2024.
- [8] F. Pan and P. Zhang, “Simulation of quantum circuits using the big-batch tensor network method,” *Physical Review Letters*, vol. 128, no. 3, p. 030501, 2022.
- [9] D. Park, H. Kim, J. Kim, T. Kim, and J. Lee, “Snuqs: scaling quantum circuit simulation using storage devices,” in *Proceedings of the 36th ACM Int’l Conf. on Supercomputing*, 2022, pp. 1–13.
- [10] B. Zhang, B. Fang, F. Ye, L. Guo, F. Song, N. Tallent, and D. Tao, “Bmqsim: Overcoming memory constraints in quantum circuit simulation with a high-fidelity compression framework,” in *Proceedings of the 39th ACM Int’l Conf. on Supercomputing*, 2025, pp. 689–704.
- [11] A. Li, B. Fang, C. Granade, G. Prawiroatmodjo, B. Heim, M. Roetteler, and S. Krishnamoorthy, “Sv-sim: scalable pgas-based state vector simulation of quantum circuits,” in *Proceedings of the Int’l Conf. for High Performance Computing, Networking, Storage and Analysis*, 2021, pp. 1–14.
- [12] C. Kim, E. Sohn, S. Kim, A. Sim, K. Wu, H. Tang, Y. Son, and S. Kim, “Scaleqsim: Highly scalable quantum circuit simulation framework for exascale hpc systems,” *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 9, no. 3, Dec. 2025. [Online]. Available: <https://doi.org/10.1145/3771577>
- [13] H. Bayraktar, A. Charara, D. Clark, S. Cohen, T. Costa, Y.-L. L. Fang, Y. Gao, J. Guan, J. Gunnels, A. Haidar *et al.*, “cuquantum sdk: A high-performance library for accelerating quantum science,” in *2023 IEEE Int’l Conf. on Quantum Computing and Engineering (QCE)*, vol. 1. IEEE, 2023, pp. 1050–1061.
- [14] Google. (2025) Qsim. Accessed: 2025-04-07. [Online]. Available: <https://quantumai.google/qsim>
- [15] G. G. Guerreschi, J. Hogaboam, F. Baruffa, and N. P. Sawaya, “Intel quantum simulator: A cloud-ready high-performance simulator of quantum circuits,” *Quantum Science and Technology*, vol. 5, no. 3, p. 034007, 2020.
- [16] X.-C. Wu, S. Di, E. M. Dasgupta, F. Cappello, H. Finkel, Y. Alexeev, and F. T. Chong, “Full-state quantum circuit simulation by using data compression,” in *Proceedings of the Int’l Conf. for High Performance Computing, Networking, Storage and Analysis*, 2019, pp. 1–24.
- [17] S. Jaques and T. Häner, “Leveraging state sparsity for more efficient quantum simulations,” *ACM Transactions on Quantum Computing*, vol. 3, no. 3, pp. 1–17, 2022.
- [18] S. Chundury, J. Li, I.-S. Suh, and F. Mueller, “Diaq: Efficient state-vector quantum simulation,” *arXiv preprint arXiv:2405.01250*, 2024.
- [19] A. Cross, “The ibm q experience and qiskit open-source quantum computing software,” in *APS March meeting abstracts*, vol. 2018, 2018, pp. L58–003.
- [20] S. Westrick, P. Liu, B. Kang, C. McDonald, M. Rainey, M. Xu, J. Arora, Y. Ding, and U. A. Acar, “Grafeyn: Efficient parallel sparse simulation of quantum circuits,” in *2024 IEEE Int’l Conf. on Quantum Computing and Engineering (QCE)*, vol. 1. IEEE, 2024, pp. 1132–1142.
- [21] L. Burgholzer, H. Bauer, and R. Wille, “Hybrid schrödinger-feynman simulation of quantum circuits with decision diagrams,” in *2021 IEEE Int’l Conf. on Quantum Computing and Engineering (QCE)*. IEEE, 2021, pp. 199–206.
- [22] S. Mandrà, J. Marshall, E. G. Rieffel, and R. Biswas, “Hybridq: A hybrid simulator for quantum circuits,” in *2021 IEEE/ACM Second International Workshop on Quantum Computing Software (QCS)*. IEEE, 2021, pp. 99–109.
- [23] W. Zhang, A. Almgren, V. Beckner, J. Bell, J. Blaschke, C. Chan, M. Day, B. Friesen, K. Gott, D. Graves *et al.*, “Amrex: a framework for block-structured adaptive mesh refinement,” *The Journal of Open Source Software*, vol. 4, no. 37, p. 1370, 2019.
- [24] P. Giannozzi, S. Baroni, N. Bonini, M. Calandra, R. Car, C. Cavazzoni, D. Ceresoli, G. L. Chiarotti, M. Cococcioni, I. Dabo *et al.*, “Quantum espresso: a modular and open-source software project for quantum simulations of materials,” *Journal of physics: Condensed matter*, vol. 21, no. 39, p. 395502, 2009.
- [25] J. Bae, J. Lee, Y. Jin, S. Son, S. Kim, H. Jang, T. J. Ham, and J. W. Lee, “[FlashNeuron]:{SSD-Enabled}{Large-Batch} training of very deep neural networks,” in *19th USENIX Conference on File and Storage Technologies (FAST 21)*, 2021, pp. 387–401.
- [26] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, and Y. He, “Zero-infinity: Breaking the gpu memory wall for extreme scale deep learning,” in *Proceedings of the Int’l Conf. for high performance computing, networking, storage and analysis*, 2021, pp. 1–14.
- [27] K. Zhou, Y. Hao, J. Mellor-Crummey, X. Meng, and X. Liu, “Valueexpert: Exploring value patterns in gpu-accelerated applications,” in *Proceedings of the 27th ACM Int’l Conf. on Architectural Support for Programming Languages and Operating Systems*, 2022, pp. 171–185.
- [28] B. Behzad, S. Prabhakar, and M. Snir, “Optimizing i/o performance of hpc applications with autotuning,” *ACM Transactions on Parallel Computing (TOPC)*, vol. 5, no. 4, pp. 1–27, 2019.
- [29] M. Khairy, V. Nikiforov, D. Nellans, and T. G. Rogers, “Locality-centric data and threadblock management for massive gpus,” in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 1022–1036.
- [30] D. Tripathy, A. Abdolrashidi, L. N. Bhuyan, L. Zhou, and D. Wong, “Paver: Locality graph-based thread block scheduling for gpus,” *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 18, no. 3, pp. 1–26, 2021.